

Name : Piyusha Sharma
IOT Assignment
IOT027

Q1) You are building a smart irrigation system that must monitor soil moisture, control a water pump, and send data to a mobile application every 5 seconds. You have both an Arduino UNO and an ESP32. Which controller would you choose and why? Would your answer change if internet connectivity was not required?

Answer: I would choose the **ESP32** because it has built-in Wi-Fi and Bluetooth, higher processing speed (240 MHz), more GPIO pins, and larger memory compared to Arduino UNO. Since the system needs to send sensor data to a mobile application every 5 seconds, internet connectivity is essential, and ESP32 is designed for such IoT applications.

The ESP32 can simultaneously:

- Read soil moisture sensor values
- Control the water pump using a relay
- Send data to cloud/mobile app
- Handle multiple tasks efficiently

If internet connectivity is **not required**, then I would choose the **Arduino UNO** because it is cheaper, consumes less power, and is sufficient for reading the soil moisture sensor and controlling the pump.

Q2) A smart factory contains 500 sensors distributed across a 2 km area. Compare Wi-Fi, Bluetooth, LoRa, and MQTT-based communication. Which protocol or combination of protocols would you choose and why?

Answer:

protocol	Range	Speed	Power consumption	Suitable use
Wifi	50-100m	High	High	Internet connection
Bluetooth	10-100m	Medium	Low	Short range communication
loRa	2-15km	Low	Very low	Long range sensor

				communication
MQTT	Depends on the network system	Efficient	Low	Cloud communication

For a factory with 500 sensors spread across 2 km, I would use **LoRa** to connect the sensors to a LoRa Gateway because it offers long-range communication with very low power consumption. The gateway would then use **MQTT** to send the collected data to the cloud.

This combination is reliable, energy-efficient, scalable, and suitable for industrial IoT systems.

Q3) A temperature sensor suddenly starts showing impossible values (e.g., -120°C or 300°C). How would you determine whether the problem lies in the sensor, wiring, power supply, ADC, or software?

Answer:

To identify the problem, I would follow these steps:

1. Check whether the sensor wiring is correct and firmly connected.
2. Measure the supply voltage using a multimeter.
3. Replace the temperature sensor with another working sensor to check whether the original sensor is faulty.
4. If using an analog sensor, observe the ADC values through the Serial Monitor.
5. Print raw sensor readings to verify whether the software is reading the data correctly.
6. Check whether the correct sensor library is being used.
7. Verify that the sensor is properly calibrated.
8. Compare the readings with another thermometer.

This systematic approach helps identify whether the issue is caused by hardware or software.

Q4) An ESP32-based weather station must operate on battery power for six months without maintenance. What hardware and software modifications would you make to maximize battery life?

Answer:

Hardware Modifications

- Use Deep Sleep mode.
- Use low-power sensors.

- Remove unnecessary LEDs.
- Use an efficient voltage regulator.
- Use a high-capacity rechargeable battery or solar charging.

Software Modifications

- Wake the ESP32 only when measurements are required.
- Upload sensor data in batches instead of continuously.
- Reduce Wi-Fi ON time.
- Disable unused peripherals such as Bluetooth.
- Use interrupt-based wake-up whenever possible.

These techniques significantly reduce power consumption and increase battery life.

Q5) An automatic braking system in a vehicle requires communication with almost zero delay. Which communication method would you choose and why?

Answer:

I would choose the CAN (Controller Area Network) **Bus**.

Reasons:

- Very low communication delay.
- High reliability.
- Built-in error detection.
- Real-time communication.
- Widely used in automotive systems.

Protocols such as Wi-Fi and Bluetooth introduce higher latency and are not suitable for safety-critical applications like automatic braking.

Q6) You need to measure the average temperature of a classroom using only one sensor. Where would you place the sensor and why? Explain how poor placement could affect the accuracy of your measurements. Assume the classroom to be a square-base cuboid of side length “a” and height “b”.

Answer:

The temperature sensor should be placed at the **center of the classroom**, approximately halfway up the height of the room.

The sensor should also be:

- Away from windows.

- Away from doors.
- Away from AC vents and fans.
- Away from direct sunlight.
- Away from projectors or heaters.

Poor placement can produce inaccurate measurements:

- Near a window → temperature may appear too high or too low.
- Near an AC vent → temperature will appear lower.
- Near the ceiling → warmer air rises, causing higher readings.
- Near electronic equipment → local heating increases temperature readings.

Therefore, placing the sensor near the center provides the most accurate estimate of the classroom's average temperature.

Q7) Under what situations could an analog sensor perform better than a digital one?

Answer:

Although digital sensors are generally more accurate and resistant to noise, analog sensors can perform better in certain situations.

An analog sensor performs better when:

- Continuous measurement is required instead of discrete digital values.
- Small changes in the measured quantity need to be detected.
- A faster response is required because no internal digital conversion is involved.
- Low-cost applications are being developed.
- Simple interfacing with an Analog-to-Digital Converter (ADC) is sufficient.

Examples:

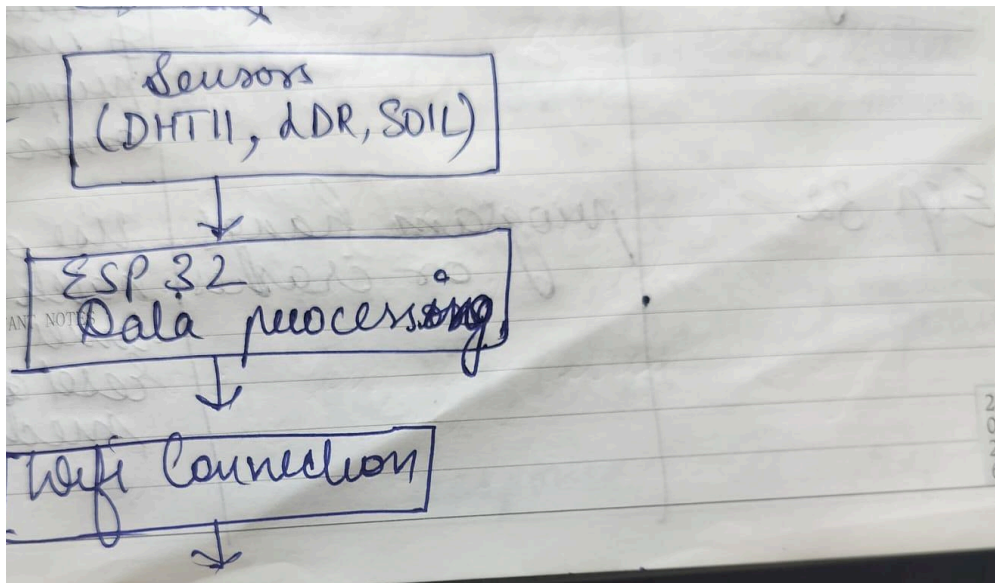
- LDR (Light Dependent Resistor) for measuring light intensity.
- Analog soil moisture sensor for detecting gradual changes in soil moisture.
- Potentiometer for position sensing.

Therefore, analog sensors are preferred when low cost, fast response, and continuous output are more important than high accuracy.

Q8) Draw the complete architecture/block diagram of your IoT project, beginning from the sensor and ending at the user interface. Identify every possible point where a failure could occur and suggest one solution for each.

Answer:

Block Diagram



17

SATURDAY

~~17-01-2026 - 017-348 - WK 03~~

APPOINTMENT MEETING

017 348 • WK 03
Internal / Router
NT/MEETING

Cloud Server
(Google Sheet / Blynk)

Mobile App/User

JANUARY

M	T	W	T	F	S	S	M	T	W	T	F	S	S	JAN
			1	2	3	4	5	6	7	8	9	10	11	
12	13	14	15	16	17	18	19	20	21	22	23	24	25	26
26	27	28	29	30	31									

failure point

Possible Problems

Schüler

Sensor

sensor damaged.

Replace
or re-
calibrate
the sensors

Weng

Loose or Broken
Connection.

check
plumbing
& use
proper
connections

18 SUNDAY

Exp 32

program hangs
or crashes

use a
"watch dog
timer"
Eset ~~and~~
mechan

IMPORTANT NOTES

2026

M T W T F S S M T W T F S S														MONDAY 19-01-2026												019-346 • WK 04 • 19-01-2026													
9 10 11 12 13 14 15 16 17 18 19 20 21 22																																							
23 24 25 26 27 28																																							
WIFI														Network Unavailable												Automatically Reconnect & Relay Stored data locally													
Internet														Internet Outage																									
Cloud Server														Server Unavailable												Relay upload data and keep backup													
App														App cannot display data												Refresh data & check API connection													

Q9) If an attacker gains access to your IoT device through the Wi-Fi network, what kinds of attacks could they perform? Suggest at least five methods to improve the security of the system (with applicable solutions, do not cite far-fetched ideas).

Answer:

If an attacker gains access to the IoT device, they could:

- Read confidential sensor data.
- Control connected devices such as relays or pumps.
- Modify sensor readings.
- Install malicious firmware.
- Launch Denial-of-Service (DoS) attacks to stop the device from functioning.

Security Improvements

1. Use **WPA2/WPA3** secured Wi-Fi with a strong password.
2. Encrypt communication using **HTTPS or TLS**.
3. Implement secure user authentication with strong passwords or tokens.
4. Regularly update the firmware to fix security vulnerabilities.
5. Disable unused ports and unnecessary services.
6. Enable **Secure Boot** so that only trusted firmware can run.
7. Use **Flash Encryption** to protect stored data.

These measures significantly improve the overall security of an IoT system.

Q10) Your ESP32 must simultaneously interface with a DHT11 sensor, an ultrasonic sensor, an OLED display, a relay module, and a soil moisture sensor. What hardware or software conflicts could occur, and how would you solve them?

Answer:

When multiple devices are connected to the ESP32, several conflicts may occur.

Possible Conflicts

- Two devices may be assigned the same GPIO pin.
- Blocking functions like `delay()` may slow down other tasks.
- The relay may draw high current and cause voltage drops.
- I²C communication issues may occur if multiple devices share the same address.
- Noise from the relay may affect sensor readings.

Solutions

- Assign different GPIO pins to every device.
- Use `millis()` instead of long `delay()` functions.
- Power the relay using a separate power supply if required.
- Check and configure I²C addresses correctly.
- Use decoupling capacitors and proper grounding to reduce electrical noise.

These solutions allow all peripherals to work together without significant conflicts.

Q11) An ESP32 uploads sensor data to the cloud every minute. The internet connection fails for 12 hours. Design a strategy that ensures no important data is permanently lost.

Answer:

To prevent data loss during an internet outage, I would use **local data storage**.

Strategy

1. Read sensor data every minute.
2. Check whether Wi-Fi is connected.
3. If Wi-Fi is available, upload data to the cloud.
4. If Wi-Fi is unavailable, save each reading with its timestamp in **SPIFFS or an SD card**.
5. Continue storing all readings until the internet connection returns.
6. Once Wi-Fi reconnects, upload the stored data in chronological order.
7. After successful upload, delete the stored records and continue normal operation.

This approach ensures that no important sensor data is permanently lost.

Q12) Your IoT device must operate continuously for five years without human intervention. What changes would you make to both the hardware and software to improve long-term reliability?

Answer:

To ensure reliable operation for five years, both hardware and software should be designed carefully.

Hardware Improvements

- Use industrial-grade sensors and electronic components.
- Use a stable and regulated power supply.
- Protect the circuit using surge protection and fuses.
- Place the device inside a waterproof and dustproof enclosure.
- Use high-quality connectors and corrosion-resistant wiring.

Software Improvements

- Enable a watchdog timer to automatically recover from crashes.
- Use OTA (Over-the-Air) firmware updates for remote maintenance.
- Store error logs for diagnostics.
- Avoid memory leaks by managing memory efficiently.
- Perform periodic self-checks of sensors and communication modules.

These improvements increase the reliability, durability, and maintenance-free operation of the IoT system over a long period.

Q13) Can an IoT system be considered "smart" if it does not use Artificial Intelligence? Justify your answer with suitable examples. What differentiates "smart" from "intelligent"?

Answer:

Yes, an IoT system can be considered **smart** even if it does not use Artificial Intelligence. A smart system is capable of sensing its environment, processing information using predefined rules, and taking automatic actions without human intervention.

For example, an **automatic street light** uses an LDR sensor to detect light intensity. When it becomes dark, the street light turns ON automatically, and when daylight returns, it turns OFF. This system is smart because it performs its task automatically, even though it does not learn or make decisions using AI.

An **intelligent** system, on the other hand, uses AI or Machine Learning to analyze data, learn from past experiences, and improve its decisions over time. For example, a smart thermostat like Nest learns a user's daily routine and automatically adjusts the room temperature to save energy.

Therefore, every intelligent system is smart, but not every smart system is intelligent.

Q14) Compare polling and interrupt-based programming in embedded systems. In which situations would interrupts significantly improve the performance of an IoT device?

Answer:

Polling and interrupt-based programming are two methods used to detect events in embedded systems

9 Polling	Interrupt
10 CPU continuously checks the device status	CPU responds only when an event occurs
11	
12 Wastes CPU time	uses CPU efficiently
1 Higher power consumption	Lower power consumption
2	
Simple to implement	Slightly more complex
4 Slower response to sudden words events	Immediate resp to events
5	
6	

Interrupts significantly improve the performance of an IoT device when:

- A push button is pressed.
- Motion is detected using a PIR sensor.
- An ultrasonic sensor receives an echo signal.
- A fire or gas sensor detects danger.
- The ESP32 wakes up from Deep Sleep.
- Immediate response is required without constantly checking the sensor.

Interrupts reduce processor workload, improve response time, and save battery power.

Q15) Every engineering design involves trade-offs. Looking at your own IoT project, identify three design decisions where you sacrificed one aspect (such as cost, speed, power, accuracy, or complexity) to improve another. Explain whether you believe those trade-offs were justified.

Answer:

In my **Room Comfort Automation** project, I made the following design trade-offs:

1. Cost vs Accuracy

I selected the **DHT11** sensor instead of the DHT22 because it is less expensive and easily available. Although the DHT11 is less accurate, it was sufficient for demonstrating temperature and humidity monitoring.

Trade-off: Lower cost, lower accuracy.

2. Simplicity vs Features

I used **Wi-Fi only** for communication instead of adding Bluetooth or LoRa. This made the project easier to build and program but reduced communication options.

Trade-off: Simpler implementation, fewer features.

3. Power Consumption vs Performance

The ESP32 remains active most of the time to continuously monitor sensors and upload data. This provides fast response but consumes more power.

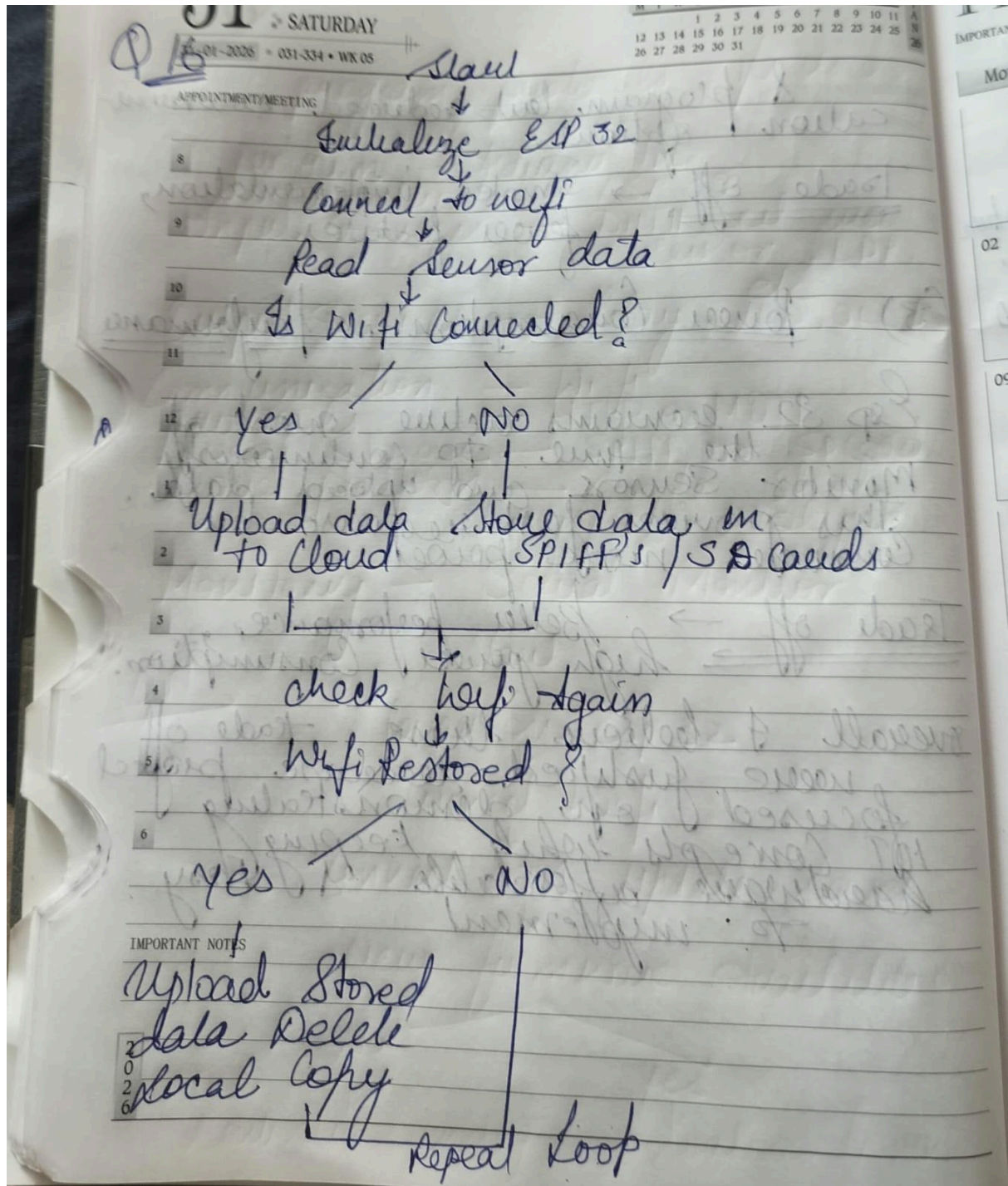
Trade-off: Better performance, higher power consumption.

Overall, I believe these trade-offs were justified because the project focused on demonstrating IoT concepts while keeping the hardware affordable and easy to implement.

Q16) Create a flowchart for an IoT system that stores sensor readings locally whenever Wi-Fi is unavailable and automatically uploads the stored data once the connection is restored. Mention all the programs/protocols/I-O used in the system.

Answer:

Flowchart



Programs, Protocols and I/O Used

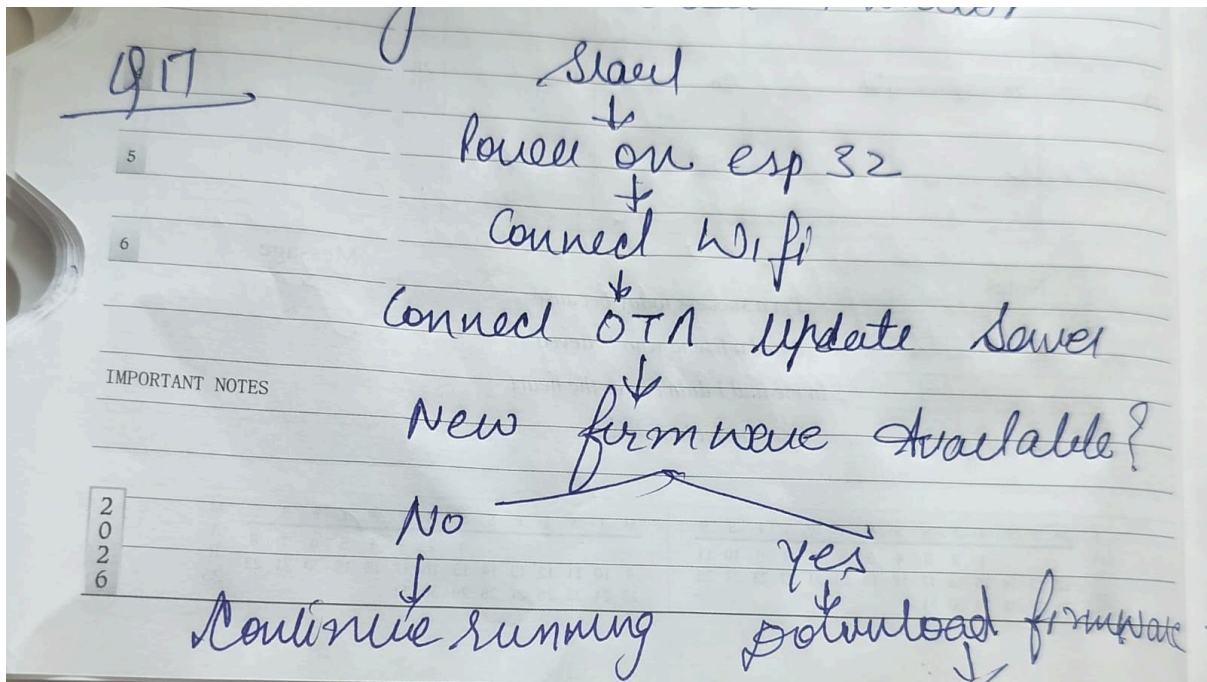
- **Program:** Arduino IDE
- **Microcontroller:** ESP32
- **Input Devices:** DHT11, Soil Moisture Sensor, LDR, etc.
- **Output Devices:** Relay, LED, OLED Display
- **Communication:** Wi-Fi
- **Protocol:** HTTP or MQTT

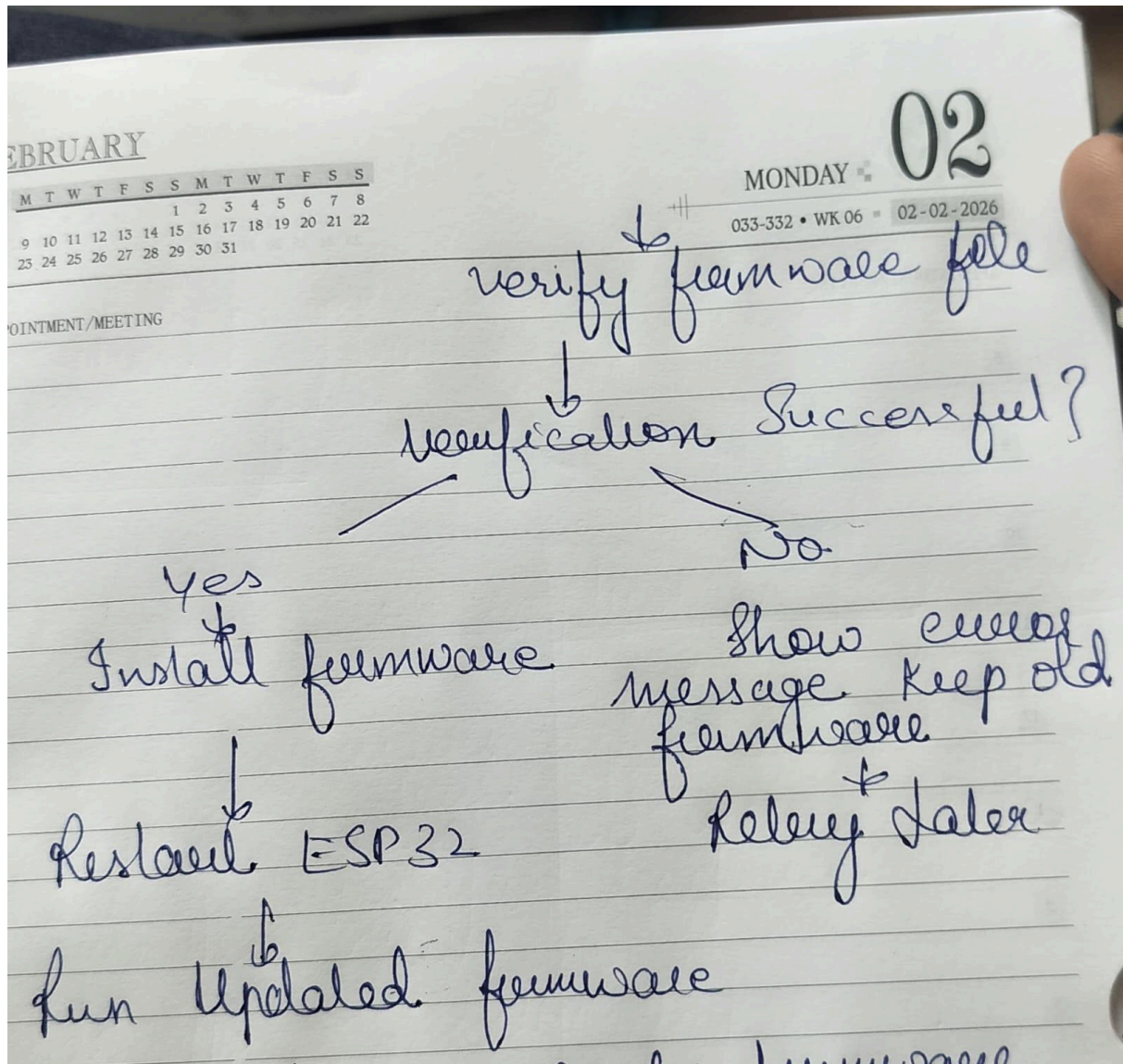
- **Local Storage:** SPIFFS or SD Card
- **Cloud Platform:** Google Sheets, ThingSpeak, or Blynk
- **Monitoring:** Serial Monitor

Q17) Draw a flowchart showing how an ESP32 performs an OTA (Over-the-Air) firmware update, including error handling if the update fails.

Answer:

Flowchart





Error Handling

- Verify the firmware before installation using checksum or digital signature.
- If verification fails, cancel the update.
- Keep the previous firmware stored as a backup.
- Restart the ESP32 with the old firmware if installation fails.
- Retry the update after a fixed time interval and log the error for debugging.

This OTA process ensures that the device can be updated remotely while preventing failures from making the system unusable.